26F STAT TA Session W12

0. Preparation for TA Session

- Download the sample code and attached datasets `earthquakes` and `offshore_drilling` (click here) and import them as `earthquake` and `oil`
  - `earthquakes` records notable earthquakes that happened in the 20th century
  - `offshore_drilling` is a survey data randomly asking registered voters in California for their position on drilling for oil and natural gas off the coast of California

1. Data Wrangling Revisited

Summarize Data and Statistics `summarize()`

- `summarize()` (or `summarise()`) creates a new dataframe of summarizing statistics

```
# example
earthquake %>%
    na.omit() %>%
    summarize(
        avg_scale = mean(richter),
        avg_death = mean(deaths)
    )

# output
 avg_scale avg_death
1 7.166379   18029.1
```

▼ **Q. What's the Difference between `mutate()` and `summarize()`?**

Ans.

`muatate()` keeps columns of the original dataframe and creates new columns (or modifies existing columns)

`summarize()` returns a new dataframe containing only the specified summary statistics

Group Data by Variables `group_by()`

- `group_by()` adds a grouping attribute to the data
- Then `dplyr` functions can perform on each group, instead of the entire data
  - Hence, we usually combo `group_by()` with `summarize()`

```
# example
earthquake %>%
    group_by(region) %>%
    summarize(
        max_scale = max(richter),
        min_scale = min(richter),
        avg_death = mean(deaths)
    ) %>%
    arrange(desc(avg_death))

# output
    region max_scale min_scale avg_death
    <chr>      <dbl>      <dbl>    <dbl>
1 Peru         7.9        7.3  350700
2 Turkmenistan 7.3        7.3  110000
3 China        8          5.9   40236.
4 Armenia      6.8        6.8   25000
5 Italy        7.2        6.5   21950.
6 Japan        8.4        6.9   20324.
7 Pakistan     8          6.2   13100
```

- We can also add the dataframe with summary statistics using `mutate()`!
- We can use `ungroup()` to drop the group structure so the latter manipulation will be perform on the entire dataframe
    - Compare the output w/ or w/o `ungroup()` !

```
# example
earthquake %>%
    group_by(region) %>%
    mutate(
        max_scale = max(richter),
        min_scale = min(richter),
        avg_death = mean(deaths)
    ) %>%
    arrange(desc(avg_death)) %>%
    ungroup()

# output
  year     deaths  region max_scale min_scale avg_death
 <int> ...  <int>   <chr>     <dbl>     <dbl>     <dbl>
1 1946 ...   1400    Peru       7.9       7.3   350700
2 1970 ... 700000    Peru       7.9       7.3   350700
3 1948 ... 110000 Turk...       7.3       7.3   110000
```

```
4 1917 ...   1800   China       8         5.9    40236.
5 1918 ...   1000   China       8         5.9    40236.
6 1920 ... 200000   China       8         5.9    40236.
7 1923 ...   3500   China       8         5.9    40236.
```

- Another example is to count the number of earthquakes in a year

```r
# example
earthquake %>%
  mutate(N = 1) %>%
  group_by(year) %>%
  summarize(earthquake_count = sum(N)) %>%
  arrange(desc(earthquake_count))

# output
   year  earthquake_count
  <int>             <dbl>
1  1976                 6
2  1992                 5
3  1906                 4
4  1923                 3
5  1931                 3
```

Problems if you Forget to `ungroup()`

- Suppose you want to find the group with the highest revenue

```r
sales <- data.frame(region = c("N","N","S","S","S","S"),
                                        store = c("A","B","A","A","B","B"),
                                        sales = c(100,120,90,110,150,140))

# forget to ungroup()
by_region_store <- sales %>%
    group_by(region, store) %>%
    summarise(total_sales = sum(sales))

#   region store total_sales
# 1 N      A             100
# 2 N      B             120
# 3 S      A             200
# 4 S      B             290
```

- Then use `filter()` to select the rows with the largest mean

```
by_region_store %>%
    filter(total_sales == max(total_sales))

# output
  region store total_sales
1 N      B             120
2 S      B             290
```

- We get two columns in the output!
  - This is because `summarize()` default to `.groups = "drop_last"` (i.e., it drops the last group `store` after the summarization)
- To avoid this problem, add `ungroup()` after `summarize()`
  - or use `summarize(..., .groups = "drop")`

```
# now you attain the correct output!
sales %>%
  group_by(region, store) %>%
  summarise(total_sales = sum(sales)) %>%
  ungroup() %>%
  filter(total_sales == max(total_sales))

# output
  region store total_sales
1 S      B             290
```

2. Proportional Table `prop.table()`

- In this section, we use the other dataset `oil`
- First look at the raw data

```
View(oil)

# output
                v1           v2  # colnames are in the 1st row...
1 position college_grad
2  support          yes
3  support          yes

# quick fix
colnames(oil) <- oil[1, ]
oil <- oil[-1, ]
```

```
# output
  position college_grad
2  support          yes
3  support          yes
```

Create Proportion Table `prop.table()`

- Recall that `table()` gives us the number counts (contingency table)

```
count_table <- table(oil$position,
                                        oil$college_grad
                                        )

# output
             no yes
do_not_know 131 104
oppose      126 180
support     132 154
```

- `prop.table()` turns a contingency table into a proportional table

```
# by row
pct_table_1 <- prop.table(count_table, margin = 1)
# by column
pct_table_2 <- prop.table(count_table, margin = 2)

# output
> pct_table_1
                     no       yes
  do_not_know 0.5574468 0.4425532
  oppose      0.4117647 0.5882353
  support     0.4615385 0.5384615

> pct_table_2
                     no       yes
  do_not_know 0.3367609 0.2374429
  oppose      0.3239075 0.4109589
  support     0.3393316 0.3515982
```

3. Hypothesis Testing `chisq.test()`

- There are three cases to use Chi-Squared test
  - **Goodness-of-Fit**: Check if the observed distribution is similar to the expected distribution

        - This is for one-way tables
    - **Test of Homogeneity**: Test if there is a similar outcome distribution among subgroups
        - This is for two-way tables
    - **Test of Independence**: Test if two variables are dependent
        - Also for two-way tables
        - e.g., the relation between living cities and supporting parties

```
# example
chisq.test(count_table) # use contingency table!

# output
    Pearson's Chi-squared test

data:  count_table
X-squared = 11.461, df = 2, p-value = 0.003246
```

- The output is stored in a list
- **Do not** use prop.table to perform Chi-Squared test! Because the Chi-Squared test needs the original counts (sample size) in the data to compute the Chi-Squared test statistics

▼ **How to Interpret the Outcome?**

- df (degrees of freedom): how many pieces of information are free to vary?
    - df of contingency table (two-way table) is $(3-1)\times(2-1)=2$ (check Ch6.4.2)
- Null hypothesis H_0: There is no association between position and colleg-degree status
- Since the p-value is less than 0.05, the outcome is significant! That is, the distribution of positions differ by college-degree status

---

Exercises

W12 Exercises